NES AI FINANCE & INSURANCE INTELLIGENCE PLATFORM

API Contracts & OpenAPI Plan

Document ID: NES-API-007 | **Version:** 1.0

Build approach: Cline-assisted development • Local-first Windows application

Scope: Personal research, paper trading and insurance document assistance

Proposed API groups, response envelopes and errors

1. API contract principles

- Define typed request/response schemas before wiring UI to real services.
- Use consistent error responses, timestamps, pagination and validation.
- Never expose provider secrets to the frontend.
- Use explicit demo adapters in the prototype; do not simulate a live provider status.

2. Proposed API groups — draft

Group	Example operations	Notes
Health	GET /api/health	App/backend health only; not market connectivity.
Market	GET /api/market/status; GET /api/market/quotes; GET /api/market/history	Provider adapter required; include source and timestamp.
Watchlists	GET/POST /api/watchlists; PATCH/DELETE item routes	Local personal records; validate symbol and exchange.
Analysis	POST /api/analysis/stock; POST /api/analysis/screener	Return inputs, outputs, source and limitations.
Backtesting	POST /api/backtests; GET /api/backtests/{id}	Include run status and assumptions.
Paper trading	GET /api/paper/portfolio; POST /api/paper/orders	Simulation only; never route to broker.

Group	Example operations	Notes
Policies	GET/POST /api/policies; GET/PATCH /api/policies/{id}	Local policy metadata; sensitive fields protected.
Documents	POST /api/documents; GET /api/documents/{id}; POST /api/documents/{id}/review	Upload, extraction state and user review.
Comparisons	POST /api/policy-comparisons	Return comparable fields and missing-data markers.
Copilot	POST /api/copilot/query	Ground response in available context; include references/limitations.
Reports	POST /api/reports; GET /api/reports/{id}	Report generation status and local file reference.

3. Standard response envelope

Success response should contain data and relevant metadata. Where appropriate, metadata includes request_id, generated_at, source, data_timestamp, status and warnings. Do not include fields that are unknown or unverified.

Example: market quote

{"symbol":"DEMO","exchange":"NSE","price":123.45,"currency":"INR","source":"DEMO_DATA","data_timestamp":null,"status":"demo","is_live":false}

Example: validation error

{"error":{"code":"VALIDATION_ERROR","message":"Review the highlighted fields.","details":[{"field":"period","issue":"Required"}]},"request_id":"example-request-id"}

4. Error handling

- Use stable error codes such as VALIDATION_ERROR, NOT_FOUND, PROVIDER_UNAVAILABLE, STALE_DATA, DOCUMENT_PARSE_FAILED and INTERNAL_ERROR.
- Return actionable but safe messages; do not expose stack traces or secrets.
- Allow retry only when appropriate and make duplicate operations safe.
- Represent unavailable timestamps as unknown/null rather than inventing values.

5. Versioning and contract tests

- Document breaking changes and version routes when necessary.
- Test schema validation, invalid requests, missing data and provider failure.
- Generate or maintain OpenAPI documentation from actual backend schemas when implemented.
- These routes are proposed contracts only; do not claim they exist until implemented.